

From Bellman Optimality to Deep Reinforcement Learning

Supplementary Theory Note for the Rotary Inverted Pendulum Laboratory

Zhixiang Ju, Haozhe Wang, and Xun Huang

This supplementary note provides the theoretical bridge that is intentionally abbreviated in the main paper: from Bellman’s optimality principle and the Hamilton–Jacobi–Bellman (HJB) equation to sampled Bellman equations and the deep-reinforcement-learning controllers used in the rotary inverted pendulum laboratory. The Bellman-to-HJB-to-reinforcement-learning reasoning was worked through step by step on the board in Class 8 before the controller experiments. The instructional purpose was not to require students to solve a nonlinear HJB partial differential equation for the rotary inverted pendulum, but to show that model-driven optimal control and modern reinforcement learning can be organized around a common recursive optimality principle. The note therefore retains the model-driven LQR/MPC branch and extends the value-based DQN derivation to the actor–critic methods PPO and TD3, so that DQN, PPO, and TD3 are not presented as disconnected algorithms.

A From Dynamic Programming to the HJB Equation

The derivation in this subsection follows the standard dynamic-programming route to the Hamilton–Jacobi–Bellman (HJB) equation [1–3]. Its role in this note is pedagogical: the goal is not to require students to solve the HJB equation for the rotary inverted pendulum, but to show how optimal control and reinforcement learning can be introduced from a common optimal-decision-making principle.

Consider a deterministic continuous-time system

$$\dot{x}(t) = f(x(t), u(t), t), \quad x(t) \in \mathbb{R}^n, \quad u(t) \in \mathcal{U}. \quad (1)$$

Here, $x(t)$ denotes the system state, $u(t)$ denotes the control input, and $\mathcal{U}$ denotes the admissible control set. For the rotary inverted pendulum laboratory, the state can be interpreted as

$$x = [\theta \quad \dot{\theta} \quad \alpha \quad \dot{\alpha}]^\top, \quad (2)$$

where θ is the rotary-arm angle and α is the pendulum angle measured from the upright equilibrium. The control input corresponds to the motor command generated by the embedded controller. In implementation, this command is constrained by the motor driver, power supply, and safety limits. A normalized admissible set can be written as

$$\mathcal{U} = \{u \in \mathbb{R} : u_{\min} \leq u \leq u_{\max}\}. \quad (3)$$

Thus, the abstract constraint $\mathcal{U}$ has a direct experimental meaning: the controller cannot command arbitrary torque or voltage, but must respect actuator saturation and safety constraints.

For a finite-horizon optimal control problem, the optimal cost-to-go function is defined as

$$J^*(t, x) = \inf_{u(\cdot) \in \mathcal{A}} \left\{ \int_t^T g(x(s), u(s), s) ds + h(x(T)) \right\}, \quad (4)$$

where T is the terminal time, $g(x, u, t)$ is the running cost, $h(x(T))$ is the terminal cost, and $\mathcal{A}$ denotes the set of admissible control trajectories satisfying $u(s) \in \mathcal{U}$. Therefore, $J^*(t, x)$ denotes the minimum remaining cost achievable from state x at time t to the terminal time T . In the RIP teaching context, $g(x, u, t)$ can be used to penalize pendulum-angle deviation, rotary-arm motion, angular velocity, and control effort.

Using a time-discretization step δ , let

$$t_k = k\delta, \quad x_k = x(t_k), \quad u_k = u(t_k). \quad (5)$$

Here, x_k and u_k denote the sampled state and sampled control input at time t_k , respectively. In the laboratory implementation, u_k corresponds to the motor command held during one sampling interval. The discrete approximation of the continuous-time optimal cost function $J^*(t, x)$ is denoted by $\tilde{J}^*(t_k, x)$.

With the first-order Euler method, the system dynamics over one sampling interval can be approximated as

$$x_{k+1} \approx x_k + f(x_k, u_k, t_k)\delta. \quad (6)$$

For compact notation, define the one-step Euler prediction from a generic state x under a generic input u as

$$\xi_k = x + f(x, u, t_k)\delta. \quad (7)$$

According to Bellman's dynamic-programming principle, the discrete approximation of the optimal cost-to-go satisfies

$$\tilde{J}^*(t_k, x) = \min_{u \in \mathcal{U}} \left\{ g(x, u, t_k)\delta + \tilde{J}^*(t_{k+1}, \xi_k) \right\}. \quad (8)$$

This expression states that the current decision should minimize the sum of the immediate cost and the optimal future cost after the state transition.

Define the two first-order correction terms as

$$A_k = \frac{\partial \tilde{J}^*}{\partial t}(t_k, x)\delta, \quad (9)$$

and

$$B_k = \nabla_x \tilde{J}^*(t_k, x)^\top f(x, u, t_k)\delta. \quad (10)$$

If $\tilde{J}^*(t, x)$ is sufficiently smooth, the second term can be expanded by a first-order Taylor approximation:

$$\tilde{J}^*(t_{k+1}, \xi_k) = \tilde{J}^*(t_k, x) + A_k + B_k + o(\delta). \quad (11)$$

Equivalently, substituting the definitions of A_k and B_k gives the expanded Taylor form without requiring a long single-line equation.

Substituting the Taylor approximation into the dynamic-programming recursion, canceling $\tilde{J}^*(t_k, x)$, dividing by δ , and letting $\delta \rightarrow 0$ gives the HJB equation

$$-\frac{\partial J^*}{\partial t}(t, x) = \min_{u \in \mathcal{U}} \left[g(x, u, t) + \nabla_x J^*(t, x)^\top f(x, u, t) \right]. \quad (12)$$

The terminal condition is

$$J^*(T, x) = h(x). \quad (13)$$

This equation shows that the optimal control law is not an isolated algebraic formula. Instead, it is obtained by balancing the instantaneous cost $g(x, u, t)$ with the change in future cost caused by the system dynamics, $\nabla_x J^*(t, x)^\top f(x, u, t)$. For students, this provides the first conceptual bridge

from Bellman’s optimality principle to continuous-time optimal control. In later subsections, linear-quadratic regulation and model predictive control are introduced as model-driven realizations of this idea, while Q-learning and deep reinforcement learning are introduced as data-driven approximations of related Bellman optimality equations.

B Sampled RIP Model: LQR and MPC

In the instructional implementation, linear quadratic regulation (LQR) and model predictive control (MPC) are not deployed directly from continuous-time formulas. Instead, both controllers are implemented on the microcontroller using a sampled-data model with the same sampling period as the embedded control loop. Let the sampling period be

$$T_s = 0.005 \text{ s}, \quad (14)$$

which corresponds to the target control frequency of 200 Hz. At the k -th sampling instant, the rotary inverted pendulum (RIP) state is written as

$$x_k = [\theta_k \quad \dot{\theta}_k \quad \alpha_k \quad \dot{\alpha}_k]^\top. \quad (15)$$

Here, θ_k is the rotary-arm angle, α_k is the pendulum deviation angle from the upright equilibrium, and $\dot{\theta}_k$ and $\dot{\alpha}_k$ are the corresponding angular velocities. In the physical platform, θ_k is obtained from the arm-angle sensor, α_k is obtained from the pendulum-angle sensor, and the velocity states are obtained either by filtered differentiation or by an observer. The input u_k denotes the normalized motor command sent to the motor driver. In firmware, this command is mapped to a pulse-width modulation (PWM) duty ratio and direction signal, and is constrained by saturation and safety limits.

Near the upright equilibrium, the nonlinear RIP dynamics are locally linearized as the linear time-invariant model

$$\dot{x}(t) = A_c x(t) + B_c u(t), \quad (16)$$

where A_c and B_c denote the continuous-time local linear model matrices obtained from modeling, identification, or course-provided parameter files. The state $x(t)$ in this local model represents deviations from the upright operating point after angle-zero calibration. Therefore, the model is intended for local stabilization, not for global swing-up.

Using zero-order-hold discretization under the sampling period T_s , the discrete-time model used by the embedded controller is

$$x_{k+1} = A_d x_k + B_d u_k, \quad (17)$$

where

$$A_d = e^{A_c T_s}, \quad B_d = \int_0^{T_s} e^{A_c \tau} B_c d\tau. \quad (18)$$

In the program implementation, A_d and B_d are computed offline by an augmented matrix exponential or by `scipy.signal.cont2discrete` in Python, and then stored in the controller parameter files. Equation (17) is the common model basis for LQR, MPC, observer design, residual-model correction, and simulation-environment construction. It is also the local linear realization of the more general prediction form $x_{k+1} = F(x_k, u_k)$ used in the Bellman and MPC discussion.

For the discrete linear model in (17), the infinite-horizon discrete LQR cost is

$$J_{\text{LQR}} = \sum_{i=0}^{\infty} \left(x_i^\top Q_x x_i + u_i^\top R_u u_i \right), \quad Q_x \succeq 0, \quad R_u \succ 0. \quad (19)$$

Here, Q_x weights the sampled RIP state and R_u weights the motor command. For example, a large weight on α_i penalizes pendulum deviation from the upright direction, a large weight on $\dot{\alpha}_i$ suppresses fast pendulum motion, and a larger R_u discourages aggressive PWM commands. Thus, the quadratic cost is not merely a textbook expression: it directly encodes the laboratory objective of keeping the pendulum upright while avoiding excessive motor effort.

For the RIP stabilization task, a quadratic cost is used because deviations from the upright equilibrium can be naturally penalized by squared state errors: for example, a larger pendulum vertical-angle deviation α_k , a larger angular velocity $\dot{\alpha}_k$, or a larger rotary-arm displacement θ_k should all produce a larger control cost, while R_u penalizes excessive motor effort.

The LQR value function is assumed to have the quadratic form

$$V(x) = x^\top P x, \quad (20)$$

where $P = P^\top \succeq 0$. From the discrete Bellman optimality equation, the discrete algebraic Riccati equation is

$$\begin{aligned} P = & Q_x + A_d^\top P A_d \\ & - A_d^\top P B_d \left(R_u + B_d^\top P B_d \right)^{-1} B_d^\top P A_d. \end{aligned} \quad (21)$$

After P is obtained, the discrete LQR gain is

$$K = \left(R_u + B_d^\top P B_d \right)^{-1} B_d^\top P A_d, \quad (22)$$

and the online feedback law is

$$u_k = -K x_k. \quad (23)$$

This Riccati-based controller is an unconstrained optimal feedback law. In the real RIP firmware, the resulting command is therefore followed by PWM saturation and safety checks before it is sent to the TB6612FNG motor driver. This distinction is important pedagogically: students learn that the mathematical LQR law gives an efficient local feedback command, whereas the embedded implementation must still respect actuator and safety constraints.

MPC can be viewed as a finite-horizon extension of LQR that explicitly includes input constraints and receding-horizon re-optimization. At sampling instant k , $x_{k+i|k}$ denotes the state predicted i steps ahead using information available at time k , and $u_{k+i|k}$ denotes the corresponding future control input. For a prediction horizon N , define

$$U_k = [u_{k|k} \quad u_{k+1|k} \quad \cdots \quad u_{k+N-1|k}]^\top. \quad (24)$$

The local linear prediction model is

$$x_{k+i+1|k} = A_d x_{k+i|k} + B_d u_{k+i|k}, \quad x_{k|k} = x_k. \quad (25)$$

Stacking the predicted states gives

$$X_k = \mathcal{A} x_k + \mathcal{B} U_k, \quad (26)$$

where X_k collects $x_{k+1|k}, \dots, x_{k+N|k}$, and the matrices $\mathcal{A}$ and $\mathcal{B}$ are determined only by A_d , B_d , and N .

The finite-horizon quadratic cost is

$$J_N = X_k^\top \bar{Q}_x X_k + U_k^\top \bar{R}_u U_k, \quad (27)$$

where

$$\bar{Q}_x = \text{blkdiag}(Q_x, \dots, Q_x, P_f), \quad (28)$$

and

$$\bar{R}_u = \text{blkdiag}(R_u, \dots, R_u). \quad (29)$$

The terminal weight P_f can be chosen as the solution of the LQR Riccati equation. Substituting the stacked prediction model yields the quadratic programming problem

$$\begin{aligned} \min_{U_k} \quad & \frac{1}{2} U_k^\top H U_k + g_k^\top U_k, \\ \text{s.t.} \quad & u_{\min} \leq u_{k+i|k} \leq u_{\max}, \\ & i = 0, 1, \dots, N-1. \end{aligned} \quad (30)$$

Here,

$$H = 2 \left(\mathcal{B}^\top \bar{Q}_x \mathcal{B} + \bar{R}_u \right), \quad g_k = G x_k, \quad (31)$$

with

$$G = 2 \mathcal{B}^\top \bar{Q}_x \mathcal{A}. \quad (32)$$

The bounds $u_{\min}$ and $u_{\max}$ correspond to the allowable normalized PWM range in the RIP experiment. If the change rate of the motor command is also important, a Δu constraint can be added. In introductory instruction, box constraints on u are used first because they are easier for students to connect to PWM saturation.

To deploy MPC on the STM32 control board, the course uses the implementation style of offline matrix generation and online lightweight solution. On the PC side, $\mathcal{A}$, $\mathcal{B}$, H , and G are precomputed and written into the firmware as C arrays. During each 5 ms control cycle, the microcontroller reads the sensor or observer state x_k , computes $g_k = G x_k$, shifts the previous optimal sequence as a warm start, and solves (30) using projected gradient descent with a fixed number of iterations. To avoid a long implementation formula, define

$$z^{(j)} = U^{(j)} - \eta \left(H U^{(j)} + g_k \right), \quad (33)$$

where η is a fixed step size. The projected-gradient update is then

$$U^{(j+1)} = \Pi_{[u_{\min}, u_{\max}]} \left(z^{(j)} \right), \quad j = 0, \dots, J_{\text{it}} - 1. \quad (34)$$

Here, $\Pi_{[u_{\min}, u_{\max}]}(\cdot)$ denotes elementwise saturation projection, and J_{it} is the fixed number of iterations. Finally, only the first control move of the optimized sequence is executed:

$$u_k = u_{k|k}^*. \quad (35)$$

PWM saturation and safety checks are applied before this command is sent to the motor driver.

Because RIP stabilization requires only a short prediction horizon and a small number of optimization variables, a fixed-iteration solver avoids unpredictable computation time and makes MPC suitable for a 200 Hz embedded control loop. Students therefore learn not only the finite-horizon optimality and constraint-handling capability of MPC, but also how an online optimization problem is transformed into deterministic-time computation on a microcontroller.

The accompanying course code is organized to make this formula-to-implementation connection explicit. The Python script for model discretization is commented with the equations defining A_d and B_d . The LQR script marks the lines corresponding to the Riccati equation and the gain

K . The MPC generation script marks the construction of $\mathcal{A}$, $\mathcal{B}$, H , and G , and the embedded firmware comments identify the projected-gradient update and the final command $u_k = u_{k|k}^*$. In the instructional release, the complete implementation code is provided through the course repository; this supplementary note focuses on the theoretical connection from the equations to the controller structures.

From this perspective, both LQR and MPC are model-driven realizations of the Bellman optimality idea in discrete sampled-data control systems. LQR obtains an explicit feedback law through the discrete Riccati equation and is extremely lightweight online. MPC implicitly defines a feedback law through finite-horizon quadratic programming and handles PWM constraints through an embedded lightweight solver. Implementing both methods on the same RIP platform helps students compare model dependence, constraint handling, computational load, and real-time behavior under identical sensing and actuation conditions.

C From HJB to the Action-Value Function

The HJB equation derived above is a finite-horizon equation, whose left-hand side is generally $-\partial J^*/\partial t$. In this subsection, to connect the HJB idea with action-value functions and Q-learning, we first consider the autonomous and stationary case. This means that the dynamics and stage cost have no explicit time dependence:

$$\dot{x} = f(x, u), \quad g = g(x, u). \quad (36)$$

The stationary setting does not require the system to be linear; a nonlinear system can also be time invariant as long as f and g do not explicitly depend on t . In this case, the value function can be written as $J_s^*(x)$, and the time derivative term in the ordinary HJB equation vanishes. The stationary HJB equation becomes

$$0 = \min_{u \in \mathcal{U}} \left[g(x, u) + \nabla_x J_s^*(x)^\top f(x, u) \right]. \quad (37)$$

Thus, the optimal action satisfies

$$u^*(x) \in \arg \min_{u \in \mathcal{U}} \left[g(x, u) + \nabla_x J_s^*(x)^\top f(x, u) \right]. \quad (38)$$

For compact notation, define

$$\mathcal{H}_J(x, u) = g(x, u) + \nabla_x J_s^*(x)^\top f(x, u). \quad (39)$$

Then (38) can be written as

$$u^*(x) \in \arg \min_{u \in \mathcal{U}} \mathcal{H}_J(x, u). \quad (40)$$

Here, $\mathcal{H}_J$ is not yet a reinforcement-learning action-value function. It still depends on the system model f , the value-function gradient $\nabla_x J_s^*$, and the solution or approximation of the HJB equation. For complex nonlinear systems, obtaining this information directly is usually difficult. To move toward a data-approximable form, an action-value function can be introduced.

In a discrete-time Markov decision process, an action is selected at one instant and then the system moves to the next state. In a continuous-time control system, however, the current control input u is itself a signal that persists over time. Therefore, to define a continuous-time action-value function, we treat u as part of an augmented state and introduce a new decision variable

$$a(t) = \dot{u}(t), \quad (41)$$

which represents the rate of change of the control command. The augmented dynamics are

$$\dot{x}(t) = f(x(t), u(t), t), \quad (42)$$

$$\dot{u}(t) = a(t), \quad \|a(t)\|_2 \leq M_a. \quad (43)$$

The constant $M_a > 0$ is the maximum admissible rate of change of the control input. In the RIP experiment, u is the normalized motor command or PWM-equivalent command, so M_a represents the largest allowed command slew rate. If $u \in [-1, 1]$ is a normalized PWM command, then M_a has units of normalized command per second, and $M_a h$ is the maximum allowed command change over a sampling interval of length h . This bound reflects motor-driver limits, firmware update limits, actuator smoothness requirements, and safety constraints.

To evaluate the future cost after fixing the current pair (x, u) , we must specify which future trajectories are admissible. Starting from $x(t) = x$ and $u(t) = u$, the future state $x(s)$ and input $u(s)$ are generated by the augmented dynamics, while the future decision is the rate signal $a(s)$. With this trajectory interpretation, the finite-horizon continuous-time action-value function is defined as

$$Q_F(t, x, u) = \inf_{\substack{a(\cdot) \\ \|a(s)\|_2 \leq M_a}} \left\{ \int_t^T g_Q(x(s), u(s), s) ds + h(x(T)) \right\}. \quad (44)$$

The admissible trajectories in (44) satisfy

$$\dot{x}(s) = f(x(s), u(s), s), \quad (45)$$

$$\dot{u}(s) = a(s), \quad \|a(s)\|_2 \leq M_a, \quad (46)$$

and

$$x(t) = x, \quad u(t) = u. \quad (47)$$

Equation (44) is the same cost-to-go idea as the finite-horizon state-value definition introduced earlier, but it is applied to the augmented state-action pair (x, u) . The state-value function asks: what is the best remaining cost if the current state is x ? The action-value function asks: what is the best remaining cost if the current state is x and the current control input is fixed at u ? The future optimization is then performed over the control-rate trajectory $a(\cdot)$. If the current input is not fixed, minimizing the action-value function over u recovers the state-value function:

$$J^*(t, x) = \min_{u \in \mathcal{U}} Q_F(t, x, u). \quad (48)$$

According to the dynamic-programming principle, for any sufficiently small interval $h > 0$, let $x_h = x(t+h)$ and $u_h = u(t+h)$. Then

$$Q_F(t, x, u) = \inf_{\substack{a(\cdot) \\ \|a(s)\|_2 \leq M_a}} \left\{ \int_t^{t+h} g_Q(x(s), u(s), s) ds + Q_F(t+h, x_h, u_h) \right\}. \quad (49)$$

Letting $h \rightarrow 0$ gives the Q-HJB equation

$$\begin{aligned} -\frac{\partial Q_F}{\partial t}(t, x, u) &= g_Q(x, u, t) + \nabla_x Q_F(t, x, u)^\top f(x, u, t) \\ &\quad + \min_{\|a\|_2 \leq M_a} \nabla_u Q_F(t, x, u)^\top a. \end{aligned} \quad (50)$$

Because

$$\min_{\|a\|_2 \leq M_a} \nabla_u Q_F(t, x, u)^\top a = -M_a \|\nabla_u Q_F(t, x, u)\|_2, \quad (51)$$

the Q-HJB equation can also be written as

$$\begin{aligned} -\frac{\partial Q_F}{\partial t}(t, x, u) &= g_Q(x, u, t) + \nabla_x Q_F(t, x, u)^\top f(x, u, t) \\ &\quad - M_a \|\nabla_u Q_F(t, x, u)\|_2. \end{aligned} \quad (52)$$

Compared with the ordinary HJB equation in (12), the Q-HJB equation in (52) evaluates the augmented pair (x, u) rather than only the state x . In (12), the optimality condition balances the stage cost $g(x, u, t)$ with the change of the state-value function along the state dynamics, $\nabla_x J^*(t, x)^\top f(x, u, t)$. In contrast, (52) contains the additional term involving $\nabla_u Q_F$, because the input u has become part of the augmented state and the new decision variable is the command rate a .

The minimization term in (50) has a useful physical interpretation. If $\nabla_u Q_F(t, x, u) \neq 0$, the minimizing control rate is

$$a^* = -M_a \frac{\nabla_u Q_F(t, x, u)}{\|\nabla_u Q_F(t, x, u)\|_2}. \quad (53)$$

Thus, the optimal rate lies on the boundary of the admissible rate set. This does not mean that every DQN action must always be the largest PWM value. Rather, it shows a boundary-seeking tendency of the continuous-time Q-HJB action-rate minimization when no additional smoothness regularization is present. In hardware experiments, steep value gradients, sensor noise, or model mismatch may therefore lead to aggressive changes of the motor command. This observation motivates noise-aware analysis, action smoothing, rate penalties, PWM saturation, and safety shutdown rules in the RIP laboratory.

A concrete RIP-oriented design example can be given through the stage cost. For the state

$$x = [\theta \quad \dot{\theta} \quad \alpha \quad \dot{\alpha}]^\top, \quad (54)$$

one may choose

$$g_Q(x, u, t) = q_\theta \theta^2 + q_{\dot{\theta}} \dot{\theta}^2 + q_\alpha \alpha^2 + q_{\dot{\alpha}} \dot{\alpha}^2 + r_u u^2, \quad (55)$$

where all weights are nonnegative. A larger q_α emphasizes upright stabilization of the pendulum, a larger $q_{\dot{\alpha}}$ suppresses fast pendulum motion, a larger q_θ discourages excessive rotary-arm displacement, and a larger r_u discourages large PWM commands. In this sense, Q_F is not a weighting matrix or a function manually selected by the instructor. It is the cost-to-go induced by the chosen RIP stage cost g_Q , terminal cost, dynamics, control-rate bound, and horizon. This is analogous to LQR, where the instructor designs state and input weights, and the value function is then determined by the Riccati equation.

D From Discounted Q-HJB to the Bellman Fixed Point and DQN

For infinite-horizon reinforcement learning, a discounted action-value function is more commonly used. To distinguish it from the finite-horizon action-value function Q_F , denote the discounted continuous-time action-value function by Q_γ :

$$Q_\gamma(x, u) = \inf_{\substack{a(\cdot) \\ \|a(t)\|_2 \leq M_a}} \left\{ \int_0^\infty e^{-\gamma t} g_{\text{RL}}(x(t), u(t)) dt \right\}, \quad (56)$$

where $\gamma > 0$ is the continuous-time discount rate. The trajectories satisfy

$$\dot{x}(t) = f(x(t), u(t)), \quad (57)$$

$$\dot{u}(t) = a(t), \quad \|a(t)\|_2 \leq M_a, \quad (58)$$

with initial conditions

$$x(0) = x, \quad u(0) = u. \quad (59)$$

For the RIP experiment, g_{RL} can use the same structure as (55), or it can add task-specific terms for swing-up, safety termination, or action smoothness. This means that the learned action-value function is not arbitrary: it is determined by the physical control objective encoded in the cost.

The corresponding discounted Q-HJB equation is

$$\begin{aligned} \gamma Q_\gamma(x, u) &= g_{\text{RL}}(x, u) + \nabla_x Q_\gamma(x, u)^\top f(x, u) \\ &+ \min_{\|a\|_2 \leq M_a} \nabla_u Q_\gamma(x, u)^\top a. \end{aligned} \quad (60)$$

Equivalently,

$$0 = g_{\text{RL}}(x, u) + \nabla_x Q_\gamma(x, u)^\top f(x, u) - M_a \|\nabla_u Q_\gamma(x, u)\|_2 - \gamma Q_\gamma(x, u). \quad (61)$$

The term $-\gamma Q_\gamma(x, u)$ comes from the exponential discount factor $e^{-\gamma t}$.

Along the optimal trajectory $(x^*(t), u^*(t))$,

$$\frac{d}{dt} [e^{-\gamma t} Q_\gamma(x^*(t), u^*(t))] = -e^{-\gamma t} g_{\text{RL}}(x^*(t), u^*(t)). \quad (62)$$

Integrating over the interval $[0, h]$ gives

$$\begin{aligned} Q_\gamma(x, u) &= \int_0^h e^{-\gamma t} g_{\text{RL}}(x^*(t), u^*(t)) dt \\ &+ e^{-\gamma h} Q_\gamma(x^*(h), u^*(h)). \end{aligned} \quad (63)$$

This expression is the Bellman relation for the continuous-time discounted action-value function.

In practical reinforcement learning, a sampling period h is used, and the control input is held constant during one sampling interval:

$$u(t) = u_k, \quad t \in [kh, (k+1)h). \quad (64)$$

For the RIP laboratory, h is chosen to match the simulation step or the embedded control period, and u_k is mapped to a discrete PWM command level. Define the one-step discounted cost as

$$c_h(x_k, u_k) = \int_0^h e^{-\gamma t} g_{\text{RL}}(x(t), u_k) dt, \quad (65)$$

and define the discrete discount factor as

$$\Gamma = e^{-\gamma h}. \quad (66)$$

Let the DQN action set be a finite set of normalized PWM commands:

$$\mathcal{U}_d = \{u^{(1)}, u^{(2)}, \dots, u^{(m)}\}. \quad (67)$$

A simple classroom example is

$$\mathcal{U}_d = \{-u_{\max}, 0, u_{\max}\}, \quad (68)$$

where the three actions correspond to negative, zero, and positive motor commands. More levels can be used when smoother action resolution is desired.

The sampled discrete-action value function satisfies the Bellman equation

$$Q_d(x_k, u_k) = c_h(x_k, u_k) + \Gamma \min_{u' \in \mathcal{U}_d} Q_d(x_{k+1}, u'). \quad (69)$$

Define the Bellman operator as

$$(\mathcal{T}Q_d)(x_k, u_k) = c_h(x_k, u_k) + \Gamma \min_{u' \in \mathcal{U}_d} Q_d(x_{k+1}, u'). \quad (70)$$

The optimal sampled action-value function is the fixed point

$$Q_d^* = \mathcal{T}Q_d^*. \quad (71)$$

Thus, Q-learning does not directly solve the continuous-time Q-HJB partial differential equation. Instead, after sampling, zero-order hold, and action discretization, it performs sample-based stochastic approximation of the discrete Bellman fixed-point equation.

For DQN [5], a neural network $Q_\theta(x, u)$ is used to approximate $Q_d(x, u)$, and a target network $Q_{\theta-}$ is used to construct the target value. For the i -th sampled transition $(x_i, u_i, c_i, x'_i, d_i)$, the target is

$$y_i = c_i + \Gamma(1 - d_i) \min_{u' \in \mathcal{U}_d} Q_{\theta-}(x'_i, u'). \quad (72)$$

Here, x_i is the sampled RIP state, u_i is the discrete PWM action applied during the sampling interval, c_i is the one-step cost, x'_i is the next state, and d_i is the termination flag. In the hardware-oriented interpretation, $d_i = 1$ can represent either the end of a simulation episode or a safety termination caused by excessive pendulum angle, excessive velocity, actuator saturation, or abnormal sensor readings.

The training loss is

$$\mathcal{L}(\theta) = \frac{1}{B} \sum_{i=1}^B \frac{1}{2} [y_i - Q_\theta(x_i, u_i)]^2. \quad (73)$$

The DQN action executed in simulation or on the microcontroller is obtained by

$$u_k^* \in \arg \min_{u \in \mathcal{U}_d} Q_\theta(x_k, u). \quad (74)$$

In the RIP experiment, the selected u_k^* is mapped to a PWM command, then passed through command saturation, optional rate limiting, and safety checks before being sent to the motor driver. If the discrete action set contains extreme PWM values and the cost does not penalize action magnitude or switching strongly enough, the greedy minimization may frequently choose boundary actions. Students therefore inspect not only reward or cost curves, but also PWM saturation ratio, command switching frequency, noise sensitivity, and safety shutdown statistics.

Therefore, DQN can be interpreted as a neural-network approximation of Bellman fixed-point iteration rather than a black-box algorithm unrelated to classical optimal control. For the rotary inverted-pendulum laboratory course, this derivation helps students understand that LQR, MPC, and DQN are different realizations of the same optimality principle under different model assumptions, computational forms, constraints, and function-approximation capabilities.

E From Bellman Evaluation to Actor–Critic Learning

DQN is a value-based method: it approximates an action-value function and obtains the action by a greedy minimization or maximization over that function. PPO and TD3 use a different computational organization. Both introduce an explicit parameterized policy (the *actor*) together with a learned value or action-value approximation (the *critic*). The Bellman principle therefore remains present, but it appears primarily in critic evaluation and in the return or advantage signal used to improve the actor [4].

To connect the cost-minimization notation above with the reward-maximization notation conventionally used in deep reinforcement learning, define the one-step reward as

$$r_k = -c_k. \quad (75)$$

Minimizing discounted cost and maximizing discounted reward are then equivalent. For this section, let s_k denote the sampled state and a_k the sampled action; on the RIP platform one may identify $s_k = x_k$ and $a_k = u_k$.

For a stochastic policy $\pi(a \mid s)$, define

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{j=0}^{\infty} \Gamma^j r_{k+j} \mid s_k = s \right], \quad (76)$$

and

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{j=0}^{\infty} \Gamma^j r_{k+j} \mid s_k = s, a_k = a \right]. \quad (77)$$

The advantage function is

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s). \quad (78)$$

The value function satisfies the Bellman expectation equation

$$V^\pi(s_k) = \mathbb{E}_{\substack{a_k \sim \pi(\cdot \mid s_k) \\ s_{k+1} \sim p(\cdot \mid s_k, a_k)}} [r_k + \Gamma V^\pi(s_{k+1})]. \quad (79)$$

If a critic $V_\psi(s)$ approximates $V^\pi(s)$, the one-step temporal-difference residual is

$$\delta_k = r_k + \Gamma(1 - d_k)V_\psi(s_{k+1}) - V_\psi(s_k), \quad (80)$$

where d_k is a termination indicator. Equation (80) is a sampled Bellman residual. This is the key link needed for actor–critic methods: even when the actor is not obtained by explicitly taking an $\arg \max$ over all actions, the critic is still trained to satisfy a Bellman consistency relation.

For a differentiable stochastic policy $\pi_\theta(a \mid s)$, the policy-gradient theorem gives the direction

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} [\nabla_\theta \log \pi_\theta(a_k \mid s_k) A^{\pi_\theta}(s_k, a_k)], \quad (81)$$

where replacing Q^π by the advantage does not change the expected gradient but reduces variance. Therefore, the conceptual chain is

$$\text{Bellman evaluation} \longrightarrow V^\pi \text{ or } Q^\pi \longrightarrow A^\pi \longrightarrow \text{policy improvement}.$$

PPO makes this stochastic policy-improvement step conservative, while TD3 uses a deterministic actor together with two Bellman-trained action-value critics.

F PPO as Bellman-Critic Evaluation Plus Conservative Policy Improvement

Proximal Policy Optimization (PPO) [6] is an on-policy actor-critic method. Let $\pi_{\theta_{\text{old}}}$ denote the policy that generated a rollout and π_{θ} the policy currently being updated. Define the probability ratio

$$\rho_k(\theta) = \frac{\pi_{\theta}(a_k | s_k)}{\pi_{\theta_{\text{old}}}(a_k | s_k)}. \quad (82)$$

A direct policy-gradient update would favor actions with positive estimated advantage and suppress actions with negative advantage. However, an excessively large policy change can destroy a useful controller in one update. PPO therefore uses the clipped surrogate objective

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_k \left[\min \left(\rho_k(\theta) \hat{A}_k, \text{clip}(\rho_k(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_k \right) \right], \quad (83)$$

where $\epsilon > 0$ limits the benefit of moving the new policy too far from the rollout policy.

The advantage estimate used in PPO is built from Bellman residuals. A common generalized-advantage estimator is

$$\hat{A}_k = \sum_{l=0}^{K-k-1} (\Gamma \lambda)^l \delta_{k+l}, \quad (84)$$

where δ_k is defined in (80), $\lambda \in [0, 1]$ controls the bias-variance tradeoff, and K is the rollout length. The critic can be trained by a squared value loss such as

$$L_V(\psi) = \frac{1}{B} \sum_{i=1}^B \left(V_{\psi}(s_i) - \hat{R}_i \right)^2, \quad (85)$$

where $\hat{R}_i$ is a bootstrapped or empirical return target. Thus, PPO does not abandon Bellman optimality. Instead, Bellman-style temporal-difference evaluation supplies the critic and advantage estimates, while the actor is improved directly through a clipped policy-gradient objective.

For the RIP task, PPO is particularly natural when the motor command is represented continuously. The policy can output the parameters of a bounded action distribution during training; at deployment, the selected action is converted to the motor command and then subjected to the same saturation, rate, timing, and safety logic as the other controllers. The essential pedagogical contrast with DQN is that DQN chooses among a finite set of actions by comparing action values, whereas PPO learns a stochastic policy and improves it through advantage-weighted probability updates.

G TD3 as Twin Bellman Critics Plus Deterministic Policy Improvement

Twin Delayed Deep Deterministic Policy Gradient (TD3) [8] is an off-policy actor-critic method for continuous actions. It builds on deterministic policy-gradient ideas [7]. The actor is a deterministic function

$$a = \mu_{\phi}(s), \quad (86)$$

and two critics, $Q_{\omega_1}(s, a)$ and $Q_{\omega_2}(s, a)$, approximate the action-value function.

For a replay-buffer transition $(s_i, a_i, r_i, s'_i, d_i)$, TD3 first constructs a smoothed target action

$$\tilde{a}'_i = \text{clip}(\mu_{\phi^-}(s'_i) + \varepsilon_i, a_{\min}, a_{\max}), \quad (87)$$

with

$$\varepsilon_i \sim \text{clip}(\mathcal{N}(0, \sigma^2), -c, c). \quad (88)$$

The Bellman target is

$$y_i = r_i + \Gamma(1 - d_i) \min_{j \in \{1, 2\}} Q_{\omega_j^-}(s'_i, \tilde{a}'_i). \quad (89)$$

Each critic minimizes

$$L_{Q_j}(\omega_j) = \frac{1}{B} \sum_{i=1}^B [Q_{\omega_j}(s_i, a_i) - y_i]^2, \quad j \in \{1, 2\}. \quad (90)$$

The minimum of the two target critics makes the target deliberately conservative and reduces the propagation of positive function-approximation errors. The target-action perturbation smooths the critic target and discourages the actor from exploiting narrow, erroneous peaks in the learned Q surface.

The actor is updated less frequently than the critics by ascending the first critic’s action value,

$$J_\mu(\phi) = \frac{1}{B} \sum_{i=1}^B Q_{\omega_1}(s_i, \mu_\phi(s_i)), \quad (91)$$

which yields the deterministic policy-gradient direction

$$\nabla_\phi J_\mu(\phi) \approx \frac{1}{B} \sum_{i=1}^B \nabla_a Q_{\omega_1}(s_i, a)|_{a=\mu_\phi(s_i)} \nabla_\phi \mu_\phi(s_i). \quad (92)$$

Target networks are then updated slowly, for example by Polyak averaging,

$$\omega_j^- \leftarrow \tau \omega_j + (1 - \tau) \omega_j^-, \quad \phi^- \leftarrow \tau \phi + (1 - \tau) \phi^-. \quad (93)$$

The Bellman connection in TD3 is therefore explicit: the two critics are trained toward a bootstrapped Bellman target, and the deterministic actor moves toward actions that those critics predict will produce higher long-term return. Compared with DQN, the continuous action no longer requires enumeration of a discrete action set. Compared with PPO, TD3 reuses off-policy replay data and performs policy improvement through the gradient of a learned action-value function rather than through a clipped likelihood ratio.

H Unified Bellman-Centered Interpretation of DQN, PPO, and TD3

The three reinforcement-learning controllers used in the laboratory can now be placed on one theoretical map. Their computational forms differ, but the common element is recursive evaluation of immediate consequences plus future value.

- **DQN:** approximates the optimal discrete-action Bellman fixed point with a neural action-value function and obtains the control command by greedy action selection. The critic-like object Q_θ directly determines the policy.
- **PPO:** represents the policy explicitly. A Bellman-trained value critic produces temporal-difference residuals and advantage estimates, and the policy is improved by a clipped stochastic policy-gradient update.

- **TD3**: represents both a deterministic continuous-action policy and two action-value critics. The critics satisfy conservative Bellman targets, while the actor is improved by differentiating through the learned action-value surface.

Accordingly, neither PPO nor TD3 should be interpreted as being outside the Bellman framework simply because their update equations look different from Q-learning. The continuous-time HJB equation provides the conceptual optimality starting point; sampling and zero-order hold lead to discrete Bellman relations; function approximation then produces value-based or actor-critic algorithms. DQN, PPO, and TD3 differ mainly in how the policy is represented, how the value information is approximated, how data are collected and reused, and how policy improvement is constrained or regularized.

For the Class 8 blackboard derivation, this unified view is the intended educational endpoint: Bellman’s principle is first used to motivate HJB and model-driven optimal control, then the sampled Bellman relation motivates action-value learning, and finally the same value-evaluation idea is extended to actor-critic learning. The later DQN, PPO, and TD3 laboratory classes can therefore be read as three implementation branches of a common optimal-decision-making framework rather than as three unrelated deep-learning recipes.

References

- [1] D. P. Bertsekas, *Dynamic Programming and Optimal Control*, 4th ed., vol. 1. Athena Scientific, 2017.
- [2] D. Liberzon, *Calculus of Variations and Optimal Control Theory: A Concise Introduction*. Princeton University Press, 2012.
- [3] D. E. Kirk, *Optimal Control Theory: An Introduction*. Dover Publications, 2004.
- [4] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.
- [5] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [6] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” arXiv:1707.06347, 2017.
- [7] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller, “Deterministic policy gradient algorithms,” in *Proc. 31st Int. Conf. Machine Learning (ICML)*, 2014, pp. 387–395.
- [8] S. Fujimoto, H. van Hoof, and D. Meger, “Addressing function approximation error in actor-critic methods,” in *Proc. 35th Int. Conf. Machine Learning (ICML)*, 2018, pp. 1587–1596.